DAY 10 TECHNICAL ACTIVITY REPORT

Competitive Programming Analysis

Algorithmic Optimization, Strategy Mapping, & Asymptotic Bounds Evaluation

TRAINEE	MENTOR	INSTITUTION	DATE & STATUS
Benatic Rithan B	Rathees G	KIT, Coimbatore Dept of CSE	19 June 2026 ● Completed Successfully

1. EXECUTIVE SUMMARY

This technical report highlights the systematic problem-solving process and efficiency benchmarks achieved during Day 10 of the competitive programming phase. The focus centered on analyzing execution limits, minimizing resource space overhead, and applying target data structures to core problems. A comprehensive set of fifteen algorithmic tasks was finalized across the **LeetCode** (10 problems) and **CodeChef** (5 problems) evaluation ecosystems. Solutions were thoroughly validated and uploaded to a remote version-controlled repository to satisfy delivery parameters.

2. DATA STRUCTURE SELECTION & COMPLEXITY ARCHITECTURE

To preserve computational safety under maximum system load, specific technical parameters and structures were matched directly to the underlying programmatic behavior of each task:

- **Dictionaries & Maps:** Leveraged within element lookups to track variable complements and history, cutting down conventional lookup iterations to constant runtime boundaries, $O(1)$.
- **Stacks (LIFO Sequences):** Implemented to maintain strict execution tracking for nested open-and-close state balance validation sequences.
- **Two-Pointer Indexes:** Applied across dynamic arrays to alter sorting sequences directly in memory, suppressing the spatial overhead down to $O(1)$ auxiliary limits.
- **Logarithmic Traversal:** Formulated split-interval boundaries to systematically drop half of the valid searchable domains during every evaluation block.

3. STRUCTURAL MAPPING MATRIX

PARADIGM CATEGORY	TARGET EVALUATION SCOPE	CORE LOGICAL STRATEGY	TARGET ASYMPTOTIC BOUNDS
Arrays & Hashing	• Two Sum • Contains Duplicate • Remove Duplicates from Sorted Array • Move Zeroes	Hash map key mappings and single-pass element frequency lookups.	$O(N)$ Runtime $O(1)$ Space (In-Place)
Two Pointers	• Merge Sorted Array • Best Time to Buy and Sell Stock	Simultaneous multi-index monitoring over active arrays.	$O(N)$ Linear Scan
Linear Structures	• Valid Parentheses	Last-In, First-Out token sequence verification.	$O(N)$ Operational Limit
Bitwise & Math	• Single Number • Palindrome Number	Bitwise XOR cancellation patterns and numerical place-value shifts.	$O(1)$ Memory Operations
Searching Paradigms	• Binary Search	Mid-point index bounding updates to cut down lookups.	$O(\log N)$ Logarithmic Time

4. TECHNICAL CODE IMPLEMENTATION

The following object-oriented code architecture shows the structural design of optimal competitive programming verification methods. The formatting has been condensed to maximize visual spatial balance:

```python
class CompetitiveSolver:
    def isValidParentheses(self, s: str) -> bool:
        stack, b_map = [], {")": "|", "}": "{", "]": "["}
        for ch in s:
            if ch in b_map:
                if not stack or stack.pop() != b_map[ch]: return False
            else: stack.append(ch)
        return not stack

    def binarySearch(self, nums: list[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] == target: return mid
            elif nums[mid] < target: left = mid + 1
            else: right = mid - 1
        return -1
```

5. WORKSPACE ENVIRONMENT STATE MOCKUPS

LEETCODE SOLVED REGISTRY COMPILATION MATRIX


```
        ["1. Two Sum", "2. Palindrome Number", "3. Valid Parentheses", "4. Merge Sorted Array", "5.
Remove Duplicates from Sorted Array", "6. Best Time to Buy and Sell Stock", "7. Contains Duplicate",
"8. Single Number", "9. Move Zeroes", "10. Binary Search"]
```


CODECHEF PROBLEM SUITE COMPILATION VECTOR


```
        [
            {"problem_name": "Add Two Number", "status": "AC", "execution_time": "0.01s",
"memory_allocated": "3.1MB"},
            {"problem_name": "Even or Odd", "status": "AC", "execution_time": "0.01s",
"memory_allocated": "3.0MB"},
            {"problem_name": "Largest Number", "status": "AC", "execution_time": "0.02s",
"memory_allocated": "3.2MB"},
            {"problem_name": "Factorial", "status": "AC", "execution_time": "0.04s",
"memory_allocated": "3.5MB"},
            {"problem_name": "Sum of Digits", "status": "AC", "execution_time": "0.01s",
"memory_allocated": "3.1MB"}
        ]
```

VERSION CONTROL TRACK METRICS (GITHUB SUBMISSION REPOSITORY STATE)

```
        Origin: remote upstream master branch
        [Commit Ref: #CP-D10-F82] feat: implement 10 optimized leetcode modules with O(1) space
tracking
        [Commit Ref: #CP-D10-F83] feat: add 5 codechef system entries with fast I/O mathematical
implementations
```

6. LEARNING OUTCOMES

- **Structural Selection Mastery:** Evaluated operational space-time bottlenecks to choose appropriate built-in structures rather than relying on nested loops.
- **Data Integrity Enforcement:** Handled runtime constraints safely by verifying input array boundaries to protect stack evaluation workflows from index overflow errors.
- **Optimized Search Logic:** Applied sorted interval partitions to dramatically cut lookup workflows down to clean logarithmic limits.
- **Multi-Platform Adaptation:** Adjusted local script execution layouts to match both the flexible input specifications of LeetCode and the high-throughput mathematical test environments of CodeChef.